Policy-Private Consent Enforcement on Public Blockchains via TPS-Assisted Zero-Knowledge Verification

Kun Li, Ankur Lohachab, Michel Dumontier, and Visara Urovi

Abstract—In blockchain-based healthcare data sharing, consent policies are expected to be enforced in a verifiable and auditable manner. However, public blockchain transparency may expose sensitive policy information through on-chain representations, verification interfaces, and observable execution artifacts. Existing privacy-preserving designs mainly protect data contents, user identities, or transaction details, but they insufficiently address the confidentiality of governing consent policies and policy-dependent leakage during enforcement.

This paper studies policy-private consent enforcement for blockchain healthcare systems. We propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI), which enables authorization decisions to be verified while limiting leakage about policy content, policy structure, requester attributes, and public-chain metadata. In TPS-ZKPI, the provider commits to a hidden consent policy on-chain and provisions policy-bound proving material to a Trusted Proof Service (TPS). The TPS constructs the witness from requester-submitted credentials and access context, generates a zero-knowledge proof of policy compliance, and the proof is verified on-chain against the policy commitment.

TPS-ZKPI adopts an explicit off-chain trust boundary for confidential policy handling: policy confidentiality is conditional on the TPS preserving the secrecy of plaintext policy material and requester authorization evidence. The design therefore relocates trust to the proof service rather than cryptographically removing it. Under this TPS trust assumption, zero-knowledge verification provides public auditability of authorization correctness without exposing the private witness to the blockchain. To reduce policy-dependent leakage through public verification artifacts, TPS-ZKPI combines fixed-length policy encoding, a reusable zero-knowledge policy interpreter, commitment-based policy binding, and public-chain transcript normalization.

We evaluate TPS-ZKPI against metadata-only adversaries that observe public-chain artifacts and conduct inference attacks on policy content, policy distinguishability, and policy structure and parameters. Under a multi-seed permutation-test criterion, no modeled supervised or unsupervised attack recovers hidden policy information above the chance baseline, and a paired gas-padding ablation attributes this protection specifically to gas-transcript normalization at a 38–40% gas overhead.

Index Terms—Blockchain, policy privacy, data use ontology, healthcare data sharing

I. Introduction

BLOCKCHAIN technology has emerged as a promising foundation for decentralized and auditable healthcare data sharing [1]–[3]. Its transparency and immutability enable verifiable consent enforcement and traceable authorization decisions, supporting cross-institutional accountability. At the same time, consent enforcement on public blockchains may expose policy-dependent contextual information, even when the underlying health data remain encrypted [4], [5].

Consent policies in healthcare are often privacy-sensitive [6]. They may encode research purposes, disease scopes, requester eligibility, institutional restrictions, ethics approval requirements, or other contextual conditions. For example, a policy based on biomedical data-use terms may reveal whether a dataset is restricted to disease-specific research, whether ethics approval is required, or whether access is limited to particular institutions or projects. If such policies are directly represented in smart contracts, embedded in verification logic, or reflected in public execution traces, unauthorized observers may infer information about the protected resource, the data owner's constraints, or the requester's authorization context.

Existing privacy-preserving blockchain systems mainly protect data contents, user identities, or transaction details [7]–[9]. Encrypted data-sharing mechanisms such as attribute-based encryption and proxy re-encryption protect access to data [10], [11], while zero-knowledge transaction systems hide asset ownership, values, or participant relationships [12]. These mechanisms are important, but they do not directly address the confidentiality of the policies that govern access decisions. In blockchain-based consent management, policy leakage may arise not only from explicit policy disclosure, but also from policy-dependent artifacts such as calldata size, public-input layout, emitted events, verifier interfaces, and gas usage. Therefore, policy-private consent enforcement requires more than hiding data or proving correctness in zero knowledge; it also requires controlling what the public-chain transcript reveals about the hidden policy.

This paper studies policy-private consent enforcement for blockchain-based healthcare data sharing under public-chain observability. The core problem is how to make consent enforcement publicly verifiable while preventing metadata-only public-chain observers from inferring the governing policy or requester authorization evidence from policy-dependent execution artifacts. In particular, an on-chain verifier should be able to check that a requester satisfies a provider-defined consent policy, while a metadata-only public-chain adversary learns minimal information from the resulting public verification transcript. We focus on Boolean-based consent policies, including policies derived from biomedical data-use conditions such

as Data Use Ontology (DUO) terms [6] and disease-scope constraints.

To address this problem, we propose the Trusted Proof Service-assisted Zero-Knowledge Policy Interpreter (TPS-ZKPI). In TPS-ZKPI, the data provider commits to a hidden consent policy on-chain and provisions the corresponding policy-bound proving material to a Trusted Proof Service (TPS). The requester submits policy-relevant authorization evidence, such as attributes, credentials, and access context, to the TPS. The TPS constructs the private witness and generates a zero-knowledge proof showing that the requester satisfies the committed policy. The proof is then verified on-chain against the provider's policy commitment and fixed-format public inputs, allowing the system to record an auditable authorization outcome without exposing the policy logic in public smart-contract execution.

TPS-ZKPI combines the following design elements. First, it uses TPS-assisted proof generation to keep confidentiality-sensitive policy material and requester evidence outside the public blockchain transcript. Second, it compiles heterogeneous Boolean-based consent policies into fixed-length Reverse Polish Notation (RPN) programs. Third, it evaluates these programs through a reusable zero-knowledge policy interpreter, avoiding policy-specific verifier contracts and supporting policy evolution without changing the public verification interface. Fourth, it applies observability-aware transcript normalization to reduce policy-dependent variation in calldata, logs, public parameters, and gas-related behavior.

The contribution is an integration for hiding policies on a public chain with public verifiability, not a new commit-and-prove primitive: commit-and-prove over a hidden witness is a standard zero-knowledge pattern, while TPS-ZKPI's focus is the system boundary, reusable fixed-operand interpreter, and metadata-aware public transcript needed for consent enforcement.

We evaluate TPS-ZKPI using attack-driven empirical analysis under a metadata-only adversarial model. The evaluation operationalizes policy privacy through three objectives: Policy Content Hiding (PCH), Policy Indistinguishability (PI), and Structure and Parameter Privacy (SPP). Supervised and unsupervised attacks are used to test whether observable blockchain artifacts allow adversaries to recover hidden policy semantics, distinguish policy families, or infer structural properties. The results show that, after transcript normalization, no modeled attack recovers hidden policy information above the chance baseline under a multi-seed permutation criterion, and a paired ablation isolates gas-transcript normalization as the mechanism responsible.

This paper makes the following contributions:

- We formulate policy-private consent enforcement as a public-chain observability problem in blockchain-based healthcare data sharing, focusing on leakage of policy content, policy distinguishability, and policy structure from public verification artifacts.
- We propose TPS-ZKPI, a TPS-assisted zero-knowledge approach that supports publicly verifiable and auditable consent enforcement while keeping policy-bound proving material and requester authorization evidence off-chain under an explicit TPS trust assumption.
- We design a reusable zero-knowledge policy interpreter for fixed-length Boolean RPN policy programs over a fixed operand space, combining policy commitment binding with a stable on-chain verification interface.
- We provide an attack-driven empirical evaluation of residual policy-dependent leakage from normalized public-chain metadata, covering supervised classification and unsupervised clustering attacks under a multi-seed permutation-test criterion.
- We conduct a paired gas-padding ablation on a public testnet that isolates transcript normalization as the mechanism responsible for the observed protection and quantifies its gas-cost overhead.

The remainder of this paper is organized as follows. Section II discusses related work and positions TPS-ZKPI against existing privacy-preserving blockchain, hidden-policy, reusable proving, and metadata-leakage approaches. Section III defines the system model, trust assumptions, adversarial model, and privacy objectives. Section IV presents the TPS-ZKPI design. Section V reports the empirical evaluation. Section VI discusses limitations and future work, and Section VII concludes the paper.

II. Related Work

Existing work addresses important fragments of policy-private consent enforcement, but not their combination under public-chain observability. Encrypted data-sharing mechanisms, such as Ciphertext-Policy Attribute-Based Encryption (CP-ABE) and proxy re-encryption (PRE) [13], [14], protect access to data. However, CP-ABE-style ciphertext policies may expose policy-related structures, while PRE-based delegation may reveal authorization patterns outside the protection target of data encryption. Transaction-privacy systems, including Zerocash-style protocols [15] and Ethereum-oriented systems such as ZETH and AZTEC [7], [8], provide public verification over private transaction witnesses, but they mainly target asset ownership, values, or participant relationships rather than compliance with a data-owner-controlled consent policy.

Hidden-policy access-control work reduces direct policy exposure by using encrypted policies, hidden-policy ABE variants, off-chain matching, or policy-holder components [13], [16], [17]. Functional encryption gives a more general cryptographic abstraction for revealing function outputs while hiding inputs [18]. Private-state blockchain systems, including Secret Network and Oasis/Ekiden, protect smart-contract state or execution through encrypted state and trusted execution environments [19], [20]. These

lines of work address adjacent forms of policy or state hiding, but TPS-ZKPI's gap is narrower: hiding consent policies on a public chain while retaining public verifiability of each authorization decision. Moreover, existing designs may still leak policy-shaped artifacts through ciphertext structures, policy-update transactions, authorization workflows, repeated interactions, or privileged policy-holder behavior, and many target permissioned or off-chain settings where the observability assumptions are weaker than those of public blockchains.

Privacy-preserving access-control and policy-evaluation mechanisms have been studied in credential-based and XACML/SAML settings [21], as well as in multi-party policy evaluation using homomorphic encryption and secure function evaluation [22]. These approaches support confidential authorization computation, but they are not primarily designed for settings where policy-evaluation artifacts are permanently and globally visible through a public blockchain. In public-blockchain consent enforcement, calldata, public inputs, emitted events, timing, and gas usage are observable to any party. In addition, these approaches do not directly provide compact evidence that a smart contract can verify against a committed, data-owner-controlled policy instance.

Reusable proving systems, including universal-setup SNARKs, composable proof systems, and zkVMs [23]–[25], improve maintainability by reducing the need for policy-specific circuits or verifier contracts. However, when deployed for public-chain consent enforcement, reusable proving alone does not guarantee policy privacy: policy-dependent program representations, public inputs, calldata layouts, logs, or gas profiles may still remain observable. This motivates TPS-ZKPI's use of a domain-specific reusable policy interpreter, which preserves proof reuse while exposing only a constrained public verification interface.

Public-chain metadata-leakage studies show that cryptographic privacy guarantees may be weakened by observable transaction-level artifacts such as timing and volume patterns [26]. Techniques such as batching, decoys, and traffic shaping can reduce some observable signals, but they are usually designed for payment privacy, anonymity sets, or transaction-linkability settings. They do not directly provide hidden-policy compliance verification or auditable consent enforcement. TPS-ZKPI incorporates this insight by treating metadata leakage as part of policy privacy and by normalizing calldata, logs, public parameters, and gas-related behavior.

Table I summarizes the positioning of TPS-ZKPI. Existing approaches protect data, transactions, authorization computation, proof reuse, or metadata in isolation. In contrast, TPS-ZKPI combines policy confidentiality, metadata-leakage control, verifiable auditability, and a stable verification interface for evolving Boolean-based consent policies.

III. Problem and Threat Model

A. System and Trust Model

We consider a blockchain-based healthcare data-sharing system in which access to protected resources is governed by provider-defined consent policies. The system involves five roles: a data provider or data owner, a data requester, a Trusted Proof Service (TPS), an on-chain access-control manager (ACM), and a resource access layer. The provider defines a consent policy P for a protected resource and registers an on-chain commitment h to the encoded policy. The requester seeks access by submitting policy-relevant authorization evidence, such as attributes, credentials, and access context. The TPS constructs the private witness and generates a zero-knowledge proof of policy compliance. The ACM verifies the proof on-chain and records the authorization outcome, while the resource access layer grants access according to the recorded decision.

TPS-ZKPI relies on the following assumptions. First, policies are provider-controlled: the provider is responsible for defining the applicable consent policy and committing the corresponding policy instance on-chain. Second, the TPS is trusted to preserve the confidentiality of policy-bound proving material and requester-submitted authorization evidence, and to follow the prescribed proof-generation workflow. Hidden policies and requester evidence are therefore protected from the public blockchain transcript, but they are visible to the TPS. Third, the commitment scheme is assumed to be hiding and binding, and the zero-knowledge proof system is assumed to satisfy completeness, soundness, and zero knowledge with respect to the implemented policy-compliance relation. Credential issuance, revocation, and real-world identity verification are treated as external functions.

The evaluated instantiation uses a provider-operated TPS. This choice avoids exposing the policy encoding or commitment-opening randomness to requesters and keeps public verification stable, but it also concentrates trust at the provider-side proof service: a compromised or colluding TPS can see both the policy and requester evidence, and a provider-operated TPS can reintroduce fabrication and stonewalling risks similar to a provider-prove design. The commitment and zero-knowledge proof machinery mainly buys public verifiability and correctness against a TPS that submits an invalid proof; it does not provide confidentiality against a compromised TPS. Reduced-trust alternatives include an independent proof service, which becomes a high-value honeypot, a TEE-backed proof service, which makes the trust assumption hardware-attested, or MPC/collaborative SNARK proof generation, where no single party sees both policy material and requester evidence.

B. Public-Chain Adversary

The primary adversary is a passive public-chain observer. The adversary can inspect smart-contract code and monitor public blockchain artifacts across multiple authorization sessions, including policy commitments,

TABLE I
Positioning of representative approaches against policy-private consent enforcement under public-chain observability. "Yes" indicates explicit support, "Partial" indicates indirect or incomplete support, and "No" indicates that the dimension is not addressed; assumptions in parentheses are part of the claim.

Approach	Policy confidentiality	Metadata-leakage mitigation	Verifiable auditability	Main limitation
Data / transaction privacy [7], [8], [13]–[15]	Partial	Partial	Partial	Protects encrypted data access or transaction semantics, but not the governing consent policy or policy-dependent authorization workflow.
Hidden-policy access control [17]	Partial	Partial	Partial	Reduces direct policy exposure, but may leak through workflows, updates, or policy-holder interactions.
MPC / HE policy evaluation [21], [22]	Yes	No	Partial	Supports confidential computation, but usually lacks public-chain-verifiable authorization evidence.
Universal SNARKs / zkVMs [23]–[25]	No	Partial	Yes	Supports proof reuse and public verification, but does not by itself hide policy-dependent public artifacts.
Metadata-leakage defenses [26]	No	Yes	No	Models or mitigates observable signals, but does not provide hidden-policy consent enforcement.
TPS-ZKPI	Yes (under TPS trust)	Yes	Yes	Provides policy-private, metadata-aware, verifiable consent enforcement under the stated TPS trust assumption.

submitted proofs, public inputs, emitted events, calldata, timestamps, and gas usage. The adversary does not observe provider-to-TPS policy material, commitment-opening randomness, requester-submitted authorization evidence, credential witnesses, or TPS-side witness construction.

Each policy-enforcement instance corresponds to an authorization session, in which a requester seeks access to a protected resource under a committed policy. For each such session, the adversary observes a public view

$$view_{\mathsf{pub}} = (h, \pi, id_{req}, id_{res}, nonce, pp, meta), \qquad (1)$$

where h is the on-chain commitment to the hidden policy, π is the zero-knowledge proof submitted for on-chain verification, id_{req}, id_{res}, and $nonce$ are the public requester, resource, and session-binding values, pp denotes other fixed-format public parameters or public handles, and $meta$ denotes transaction-level metadata exposed by the enforcement layer, such as calldata footprint, event structure, public-input layout, timing, and gas usage. The adversary's goal is to infer hidden information about the committed policy or the authorization interaction from this public view.

In addition to the public-chain observer, we treat the requester as untrusted with respect to policy confidentiality. The requester may initiate access requests, observe its own submitted evidence and authorization outcome, and inspect the public transcript. However, the requester does not receive the encoded policy, commitment-opening randomness, or policy-bound proving material, which remain within the provider-to-TPS proof-generation boundary.

Compromised TPS behavior, adaptive requester probing, and off-chain side channels are non-goals for the current model. The SNARK checks internal consistency of the witness it is given; it does not prove that the private attribute vector is the requester's real-world attribute set. Binding a proof to id_{req} ties the proof to an identifier and session context, but without an issuer-verification layer it does not by itself bind every private attribute value to an external source of truth.

C. Privacy Objectives

We define policy privacy through three attack-oriented objectives.

Policy Content Hiding (PCH). PCH requires that public-chain artifacts should not reveal the semantic content of the hidden consent policy, such as data-use terms, disease categories, research purposes, institutional restrictions, or ethics-related requirements.

Policy Indistinguishability (PI). PI requires that different candidate policies should not become distinguishable from their public verification artifacts when they are compiled into the same fixed public interface. This captures whether two policies can be told apart from commitments, proofs, public parameters, or metadata even when their direct representations are hidden.

Structure and Parameter Privacy (SPP). SPP requires that public-chain artifacts should not reveal structural or parametric properties of the hidden policy, such as logical structure, nesting depth, predicate count, set size, or threshold-like parameters.

D. Problem Statement

Given a provider-defined consent policy P, requester authorization evidence, an untrusted public blockchain, and the stated TPS trust assumption, the problem is to enforce access to a protected healthcare resource while satisfying three requirements. First, authorization must be publicly verifiable: the ACM should be able to verify that the requester satisfies the committed policy without learning or executing the policy in plaintext. Second, policy-dependent information should remain hidden from public-chain observers, including policy content, policy distinguishability, structural properties, requester evidence, and metadata-level enforcement fingerprints. Third, authorization outcomes should remain auditable, so that

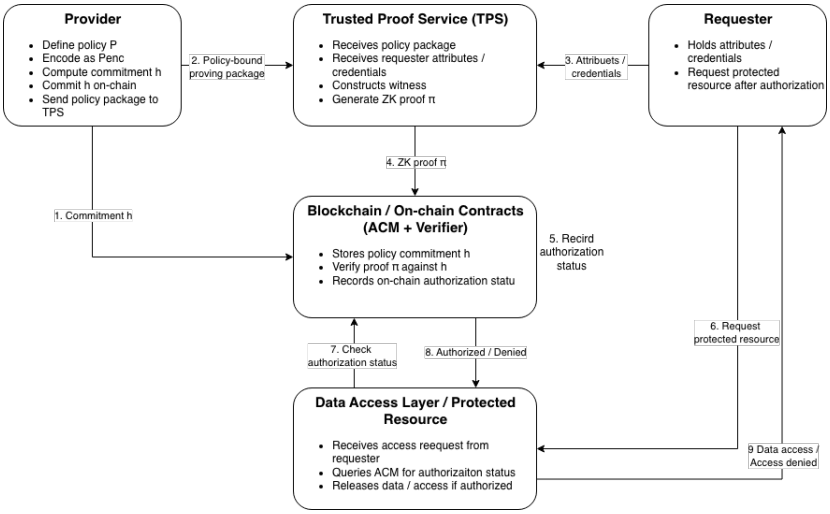


Fig. 1. TPS-assisted policy-compliance workflow.

successful access decisions can be recorded and checked without exposing the hidden policy logic.

IV. TPS-ZKPI Design

A. Overview

TPS-ZKPI follows a Trusted Proof Service-assisted architecture for policy-private and auditable consent enforcement. As shown in Figure 1, the workflow binds a provider-committed hidden policy to requester-submitted authorization evidence through TPS-side proof generation and on-chain proof verification. The provider registers only a policy commitment on-chain and sends policy-bound proving material to the TPS off-chain. For each access request, the TPS constructs the private witness from the hidden policy material and requester evidence, generates a zero-knowledge proof of policy compliance, and submits a fixed-format proof statement for ACM verification. The ACM records the authorization outcome, which is then consumed by the resource access layer.

This design separates confidentiality-sensitive proof generation from public verification. Policy material and requester evidence remain within the provider–TPS–requester off-chain boundary, while the blockchain verifies only a fixed-format proof statement against the provider's policy commitment and session-binding public inputs. Compared with a requester-prove design in which the requester receives P_{enc} and the commitment-opening randomness, TPS-ZKPI closes that certain leak to the requester; the corresponding confidentiality failure now occurs only if the TPS trust boundary is violated.

B. Policy Representation and Interpretation

TPS-ZKPI represents heterogeneous Boolean-based consent policies in a fixed-length, circuit-friendly format. Let

$$X = \{0,1\}^N, \qquad B = \{0,1\}^M \tag{2}$$

denote the requester-attribute assignment space and the external-predicate assignment space, respectively. The full assignment space is

$$O = X \times B, \tag{3}$$

where $a \in X$ encodes requester attributes and $b \in B$ encodes externally attested predicate outcomes, such as credential validity, ontology-linked checks, or obligation-related conditions. TPS-ZKPI does not itself solve credential issuance, revocation, or external attestation; it consumes these outcomes and, where applicable, checks auxiliary witnesses inside the proof relation. In the current circuit implementation, issuer-signature verification is represented by a hash-based placeholder gadget rather than a full EdDSA verifier, so attribute-origin binding is not enforced in-circuit.

Consent policies are expressed as Boolean formulas over assignment bits:

$$P ::= atom \mid (P \wedge P) \mid (P \vee P) \mid \neg P, \qquad atom ::= x_i \mid b_j. \tag{4}$$

Here x_i selects requester-attribute bit a_i and b_j selects external-predicate bit b_j. Each policy is compiled into a Reverse Polish Notation (RPN) program over operand tokens, Boolean operator tokens, and deterministic no-op tokens. To support reusable verification, each program is padded to a fixed token length L:

$$P_{enc} = [t_1, \ldots, t_L], \tag{5}$$

where shorter policies are padded with no-op tokens. The implemented circuit uses $L = \mathsf{DEPTH} = 20$; the name DEPTH is an implementation parameter for RPN token count, not formula nesting depth. No-op tokens are ignored by the stack semantics and leave the final output unchanged. We denote the set of length-L programs by Prog_L.

The reusable policy interpreter is a zero-knowledge circuit

$$C : \mathsf{Prog}_L \times O \to \{0,1\}. \tag{6}$$

Given an encoded policy and operand values, the interpreter evaluates the RPN program using deterministic stack semantics and enforces well-formedness constraints inside the circuit. The circuit outputs 1 only if the program is syntactically valid and the requester evidence satisfies the encoded policy. An output of 0 is not diagnostic: it may mean either a malformed program or unsatisfied evidence. The proved acceptance relation is therefore the asserted case $C(P_{enc}, (a, b)) = 1$. This interpreter is a fixed-operand stack-based RPN evaluator, not a general policy language runtime.

This design avoids policy-specific circuits and verifier contracts while supporting policy evolution through different fixed-length policy programs.

C. Commitment Binding and Proof Generation

To bind each proof to a provider-defined hidden policy, the provider samples commitment randomness r uniformly from the scalar field and computes

$$h = \mathsf{Com}(P_{enc}; r) = \mathsf{Poseidon}(P_{enc} \parallel r), \tag{7}$$

where h is stored on-chain as the public handle of the hidden policy. The commitment binds future proofs to the committed policy instance under the collision resistance of Poseidon. Hiding is an explicit commitment assumption that relies on high-entropy r and the fixed-length encoding of P_{enc}, which keeps the sponge input unambiguous.

For an access request, the TPS constructs the private witness

$$w = (P_{enc}, a, b, r, \omega), \tag{8}$$

where P_{enc} is the encoded policy, a denotes requester attributes, b denotes external predicate outcomes, r is the commitment randomness, and ω denotes auxiliary witnesses for credential- or predicate-dependent checks. The public statement contains the policy commitment h, requester and resource identifiers, a session nonce, and fixed-format public parameters pp.

For the compiled circuit and Groth16 common reference string, setup produces a proving key and verification key:

$$(pk, vk) \leftarrow \mathsf{Setup}(C). \tag{9}$$

The TPS generates a proof

$$\pi \leftarrow \mathsf{Prove}\big(pk, w; (h, id_{req}, id_{res}, nonce, pp)\big). \tag{10}$$

The proof certifies the conjunction that $C(P_{enc}, (a, b)) = 1$, the hidden policy encoding is consistent with the on-chain commitment h, required external-predicate checks are internally consistent with respect to the supplied parameters and placeholder gadgets, and the proof is bound to the requester, resource, and session context.

The ACM verifies π on-chain against vk and the public statement. If verification succeeds, it records a positive authorization outcome for the requester and resource. The policy encoding, commitment randomness, requester attributes, credential witnesses, and auxiliary proof material are not posted on-chain.

D. Transcript Normalization

Zero-knowledge proofs hide private witness values, but surrounding blockchain artifacts may still leak policy-dependent information. TPS-ZKPI therefore normalizes the public transcript so that different policies induce fixed or tightly bounded observable profiles. Table II summarizes the mechanisms.

TABLE II
Transcript-normalization mechanisms in TPS-ZKPI.

Artifact	Normalization mechanism
Calldata	Fixed verification interface and padded proof/public-input encoding.
Logs	Fixed event structure with constant topic and payload layout.
Public parameters	Fixed-format handles instead of policy artifacts or credential witnesses.
Gas usage	Policy-independent verifier logic with bounded or padded gas profiles.

These mechanisms do not eliminate all public observability. Rather, they reduce policy-dependent variation in calldata, logs, public parameters, and gas-related behavior that could otherwise serve as a fingerprint for the metadata-only adversary defined in Section III. TPS-ZKPI does not flatten all residual channels: timing between transactions, transaction ordering, mempool or broadcast patterns, and off-chain TPS communication remain outside the normalized public transcript.

V. Privacy Evaluation

We evaluate the three privacy properties of our system—Policy Content Hiding, Policy Indistinguishability, and Structure & Parameter Privacy—against a passive on-chain observer (Section V-A). The primary evidence is a supervised, multi-seed attack evaluation with policy-disjoint splits and permutation tests (Section V-B); unsupervised clustering provides a label-free cross-check on the same executions. All three properties pass the supervised criterion, and the clustering attack finds no natural grouping by hidden labels.

A. Adversary and observables

We evaluate against a passive on-chain observer: an adversary who sees every transaction and event our protocol emits, but never the plaintext policy, the witness, or any client-side timing. This is the strongest realistic attacker for a public blockchain—it has unrestricted, perpetual read access to the ledger.

Concretely, for each policy execution (a commit transaction followed by an access transaction) the adversary observes the following nine on-chain features:

- Gas: commitGasUsed, accessGasUsed.
- Calldata size: commitCalldataLength, accessCalldataLength.
- Proof size: proofSize.
- Event logs: commitLogsCount, commitLogsSize, accessLogsCount, accessLogsSize.

- Timing: blockDifference (blocks between commit and access).

By design, calldata length, proof size, and log counts/sizes are content-independent (constant across policies), so any residual signal can only flow through the gas features in the modeled public transcript. The defenses (commitment-only interface, randomized access-gas padding) target exactly this channel. We do not claim to flatten timing, transaction ordering, mempool or broadcast patterns, or off-chain TPS communication; compromised TPS behavior, adaptive requester probing, and off-chain side channels remain outside this evaluation model, matching the system model in Section III.

We consider three increasingly fine-grained attacker tasks, one per privacy property: (i) recover the policy's content (PCH, Section V-C); (ii) tell two semantically related policies apart (PI, Section V-D); and (iii) recover the policy's structure or parameter magnitude (SPP, Section V-E). The primary attacker is supervised and label-aware; a separate unsupervised clustering attacker provides a label-free cross-check (Section V-B).

B. Attack methodology

We evaluate each privacy property with a primary supervised attack and a complementary unsupervised cross-check. The supervised pipeline asks whether a trained classifier can recover a hidden label from the observables; this is the main privacy verdict because it gives the adversary labels and therefore more power than a purely exploratory analysis. The unsupervised pipeline asks whether the observables cluster along the hidden label without any supervision, providing a label-free sanity check on the same executions.

1) Supervised attacks: For every task we train linear and non-linear classifiers and report the score used by the unified verdict for that property. Each task is evaluated over 30 random seeds using a policy-disjoint train/test split: the policies used to train the attacker never appear in the test set, so a high score cannot come from memorizing individual policies. For each seed we run a label-permutation test and record whether the AUC is significant at α=0.05.

The honest headline metrics are the AUC, its sample-estimated distance from chance $|\text{AUC} - 0.5|$ (reported as the AUC gap), and the permutation p-value. This AUC gap is an empirical diagnostic, not the advantage of a formal cryptographic distinguisher and not a proven security bound. Raw accuracy gain is retained only as a legacy diagnostic field because, on imbalanced test sets, a majority-class predictor can score large apparent gains with $F_1 \approx 0$. The unified verdict (experiments/core/multiseed_criterion.js) is:

> FAIL iff $\text{AUC}_{\text{mean}} > 0.55$ and the significant-seed fraction is ≥ 0.5 (over 30 policy-disjoint seeds); PASS otherwise.

Both legs are required: high classification power that is not statistically reproducible across seeds (or reproducible significance with chance-level AUC) does not constitute a leak. Subclasses whose median test size is $n < 20$ are underpowered—reported but excluded from gating, so a tiny subclass's noise cannot decide the verdict. The result tables report the AUC selected by the committed verdict file for each task. SPP uses the calibrated random-forest score for its AUC leg because logistic regression over-separates on the near-constant SPP features and yields a degenerate AUC.

The LR/RF feature set is deliberately small because the transcript normalization fixes calldata footprint, proof size, public-input layout, and event-log shape by construction. The live modeled channel is therefore gas, which the on-chain observer sees completely. To distinguish a genuine defense from a weak attacker, we pair the near-chance main results with the gas-normalization positive control in Section V-H: the same features, splits, and classifiers recover signal on the un-padded gas transcript and collapse after padding.

2) Unsupervised clustering attacks: The clustering attacker drops the labels entirely and asks whether the gas features naturally group along the hidden property. We run K-Means (with K set to the number of classes, or K=2 for policy pairs) and DBSCAN on the on-chain gas features, and score the recovered clusters against the true labels with the Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI). Values near 0 mean the clusters carry no information about the hidden label; values near 1 mean full leakage.

PCH clusters all executions globally (no per-layer splitting), so the large-sample ARI/NMI are read directly. For the PI pairs we additionally guard against two small-sample failure modes.

a) Controlling sample-size confounds.: The naive PI pair construction produced severely imbalanced pairs (one policy with 1 execution against another with up to 30), where K-Means cannot cut the data and ARI/NMI ≈ 0 is an artifact rather than evidence of privacy. We therefore rebuild the PI pairs from policies with ≥ 30 executions into balanced 30-vs-30 contrasts before clustering. To rule out residual finite-sample inflation, we add a permutation baseline: we shuffle the pair labels 200 times and recompute the clustering NMI, giving the NMI distribution under zero true signal. The debiased NMI is the observed value minus this baseline, and the empirical p-value is the fraction of permutations whose NMI meets or exceeds the observed; a pair leaks only if its debiased NMI is positive and significant ($p < 0.05$).

b) Dataset.: Both attack families run on the same fixed-circuit, commitment-only, access-gas-padded Polygon Amoy dataset: the supervised verdicts read experiments/multiseed_results/polygonAmoy/ (numbers of record), and the clustering reads the same executions exported to python/data/polygonAmoy/. The supervised and unsupervised attacks therefore constitute two independent attack families applied to one dataset, rather than two datasets. Per-task sample sizes vary and are listed in the result tables (e.g. 2005 DUO executions; 133 balanced

TABLE III
Headline privacy results on the Polygon Amoy dataset. The supervised attacker uses 30 policy-disjoint seeds and a permutation test. A task passes unless both $AUC_{mean} > 0.55$ and at least 50% of seeds are significant.

Property	Hidden target	Headline supervised result	Verdict
PCH	DUO terms and ICD-10 chapters	16 content-recovery tasks pass; DUO AUCs 0.490–0.527, ICD AUCs 0.458–0.573	✓
PI	Related policy-pair identity	Overall AUC 0.516 ± 0.030, permutation $p = 0.367$, 10% significant seeds	✓
SPP	Structure, predicate, and parameter buckets	18 pairwise contrasts pass; AUCs 0.428–0.521, max 27% significant seeds	✓

TABLE IV
Policy Content Hiding: per-term supervised attack over 30 policy-disjoint seeds. Each DUO term is a one-vs-rest binary task. %sig is the fraction of seeds whose permutation test rejects at α=0.05. All six tasks pass the unified criterion.

Term	AUC (mean ± std)	perm p	%sig	med n	verdict
DUO GR	0.497 ± 0.024	0.535	0%	613	✓
DUO DS	0.490 ± 0.021	0.624	3%	613	✓
DUO RS	0.527 ± 0.019	0.191	13%	613	✓
DUO POA	0.493 ± 0.023	0.579	0%	613	✓
DUO GS	0.521 ± 0.018	0.251	13%	613	✓
DUO CC	0.501 ± 0.018	0.479	0%	613	✓
Overall: ✓ PASS					

TABLE V
Policy Content Hiding: per-chapter ICD-10 supervised attack over 30 policy-disjoint seeds (one-vs-rest, ICD-bearing rows only; median test $n = 300$). The highest value is ch6 (AUC 0.573, %sig 17%), but it clears neither the AUC nor the reproducibility leg of the criterion, so all ten chapters pass.

Chapter	AUC (mean ± std)	perm p	%sig	med n	verdict
ICD ch0	0.488 ± 0.068	0.536	0%	300	✓
ICD ch1	0.490 ± 0.050	0.560	0%	300	✓
ICD ch2	0.513 ± 0.051	0.424	7%	300	✓
ICD ch3	0.524 ± 0.042	0.358	0%	300	✓
ICD ch4	0.547 ± 0.055	0.278	17%	300	✓
ICD ch5	0.458 ± 0.060	0.700	0%	300	✓
ICD ch6	0.573 ± 0.061	0.206	17%	300	✓
ICD ch7	0.490 ± 0.048	0.595	0%	300	✓
ICD ch8	0.537 ± 0.055	0.327	10%	300	✓
ICD ch9	0.524 ± 0.057	0.395	7%	300	✓
Overall: ✓ PASS (all 10 chapters)					

30-vs-30 PI pairs); the PI debiasing averages each observed NMI over 20 K-Means seeds against a 200-permutation baseline. Provenance for every table and figure is listed in paper/README.md.

C. Policy Content Hiding (PCH)

a) Property.: PCH requires that the on-chain observer cannot recover the content of a policy—its DUO data-use terms or the ICD-10 disease codes it references—better than chance.

b) Scope of the labels.: We do not attack the full Data Use Ontology and its linked ontologies. The complete DUO has 23 data-use terms, and ICD-10 has tens of thousands of individual codes; most never appear in our generated policies, and those that do are too rare to support a statistically powered one-vs-rest attack. We therefore evaluate the six DUO terms that occur with enough support in the dataset (GR, DS, RS, POA, GS, CC; smallest class n=30) and aggregate disease codes to the ten ICD-10 chapters. This is the right granularity for the privacy question—the observer cares whether the category of permitted use or disease leaks, not which leaf code is used—and it is also a conservative choice: attacks on coarser, better-supported labels are easier than on rare leaf terms, so passing here does not understate the leak. The defenses (commitment-only interface, gas padding) are content-agnostic and apply identically to the terms we do not enumerate.

c) Supervised attack.: We treat each of the six DUO terms (GR, DS, RS, POA, GS, CC) as a one-vs-rest binary recovery task and run the multi-seed pipeline of Section V-B. Table IV reports the authoritative per-term verdicts: every DUO task has $AUC_{mean} \in [0.490, 0.527]$ and a significant-seed fraction of at most 13%, so none approaches either leg of the failure criterion. The same pipeline applied to the ten ICD-10 chapters (one-vs-rest over the ICD-bearing rows only; Table V) passes all ten. The single highest value is ch6 at $AUC_{mean} = 0.573 \pm 0.061$ with 17% significant seeds; it clears the AUC leg ($0.573 > 0.55$) but not the reproducibility leg ($17\% < 50\%$), so it is a boundary case rather than a stable channel. We report such cases rather than suppress them.

D. Policy Indistinguishability (PI)

a) Property.: PI strengthens PCH: even when two policies are semantically related, the observer must not tell which one was executed—formally, $\Pr[\hat{b} = b] \approx \frac{1}{2}$. We test four pair archetypes: semantic_equivalent (identical normalized logic), structural_variant (same semantics, different execution protections), weak_equivalent (≥ 0.95 semantic overlap), and controlled_difference (small targeted deltas; a regression control).

b) Supervised attack.: The distinguisher's task is to decide, from the on-chain features of a single execution, which policy of the pair produced it. We evaluate this single-sample classifier over 30 policy-disjoint seeds (Table VI). The overall AUC is 0.516 ± 0.030 with permutation $p = 0.367$ and only 10% of seeds significant. All three gating archetypes—semantic_equivalent, structural_variant, and weak_equivalent—remain within noise of chance. controlled_difference is reported as a diagnostic regression control but does not gate the PI verdict.

c) Why multi-seed matters.: Earlier three-seed diagnostics made the structural_variant archetype look separable, but the 30-seed policy-disjoint run in Table VI

TABLE VI
Policy Indistinguishability: single-sample distinguisher (Level I) over 30 policy-disjoint seeds, by policy-pair archetype. controlled_difference is reported as a diagnostic regression control and excluded from the PI verdict.

Pair archetype	AUC (mean ± std)	AUC gap (mean ± std)	perm p	%sig	med n	verdict
overall	0.516 ± 0.030	0.026 ± 0.022	0.367	10%	270	✓
semantic_equivalent	0.511 ± 0.082	0.070 ± 0.045	0.456	13%	71	✓
structural_variant	0.524 ± 0.071	0.059 ± 0.046	0.407	0%	40	✓
weak_equivalent	0.494 ± 0.085	0.069 ± 0.051	0.529	7%	54.5	✓
controlled_difference	0.510 ± 0.076	0.065 ± 0.041	0.452	3%	68	– (diagnostic)
Overall: ✓ PASS						

gives AUC 0.524 ± 0.071 with 0% significant seeds. This is precisely the failure mode the multi-seed criterion of Section V-B is designed to reject: a single split can create an apparently meaningful spike that does not reproduce across independent policy splits. The gas-normalization positive control in Section V-H shows that the same PI attack pipeline does recover signal before padding, so the residual structural-variant fluctuation is a measured limit of the normalized transcript rather than evidence of an incapable attacker.

E. Structure & Parameter Privacy (SPP)

a) Property.: SPP requires that neither the shape of a policy (logical structure, nesting depth, predicate coverage) nor the magnitude of its parameters (whitelist/set sizes) leaks to the observer. The supervised attack organizes the test along three annotation axes—logical structure type, predicate count, and parameter magnitude—and attacks each pair of buckets within an axis.

b) Supervised attack.: Table VII reports all 18 pairwise contrasts over 30 policy-disjoint seeds across structureType, predicateCombo, and paramType. AUC is the calibrated random-forest score, since logistic regression over-separates on the near-constant SPP features. Every contrast has $\text{AUC}_{\text{mean}} \in [0.428, 0.521]$ and a significant-seed fraction at or below 27%, so all 18 pass and SPP passes overall. We gate on AUC and the permutation test. Single-split accuracy-style diagnostics can run large on imbalanced contrasts, but this is the majority-class artifact described in Section V-B—a near-constant predictor scoring apparent gains without genuine separation—which the AUC-plus-permutation criterion correctly reads as chance. Section V-H provides the corresponding positive control: when gas padding is removed, the same random-forest SPP attacker recovers structural and parameter-magnitude signal, and that signal collapses under the full normalization.

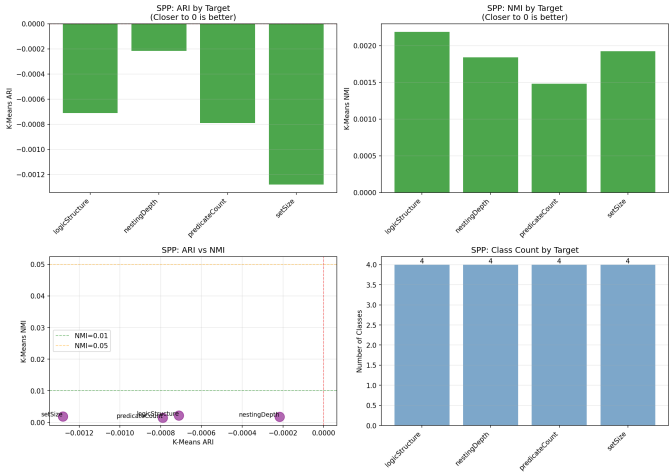


Fig. 2. SPP unsupervised clustering: ARI and NMI by structural target. All four targets sit at the zero line—gas clusters carry no structure or parameter information; NMI is read against Section V-G.

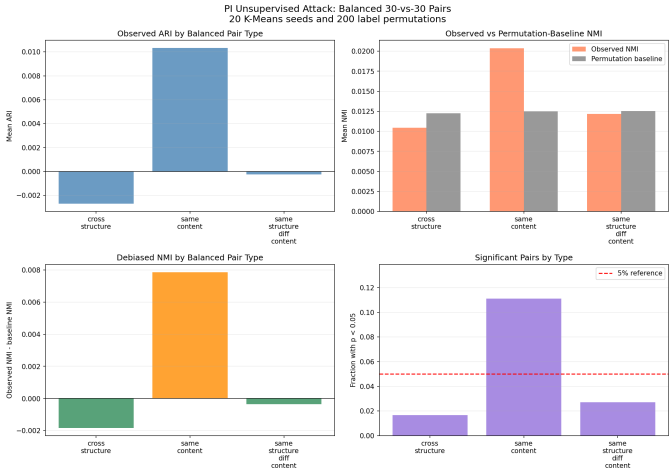


Fig. 3. PI unsupervised clustering by pair archetype: per-pair ARI/NMI stay at chance across all archetypes, matching the supervised distinguisher and the NMI null sensitivity analysis in Section V-G.

F. Unsupervised clustering: a label-free cross-check

The supervised attacks (Sections V-C–V-E) assume an adversary who can train on labeled executions. A weaker but assumption-free attacker simply asks whether the on-chain gas features cluster along the hidden property with no supervision at all. Table VIII consolidates this attack across all three properties: K-Means recovers essentially nothing, with ARI and NMI within ±0.02 of zero for every DUO, PI, and SPP target. NMI values are interpreted against the null finite-sample simulation in Section V-G. The unsupervised attacker agrees with the supervised verdict—privacy holds—as a second, label-free attack family on the same dataset (Figures 2 and 3).

TABLE VII
Structure & Parameter Privacy: 18 pairwise contrasts across three annotation axes, each over 30 policy-disjoint seeds. AUC is the calibrated random-forest score. Every contrast satisfies $\text{AUC}_{\text{mean}} \leq 0.55$ and %sig $< 50\%$.

Contrast	AUC (mean ± std)	AUC gap (mean ± std)	perm p	%sig	med n
structureType					
and_only / or_only	0.475 ± 0.051	0.036 ± 0.029	0.547	10%	102
and_only / negation_heavy	0.428 ± 0.058	0.060 ± 0.028	0.537	23%	90
and_only / and_or_mixed	0.521 ± 0.073	0.057 ± 0.041	0.432	27%	78
or_only / negation_heavy	0.440 ± 0.061	0.049 ± 0.030	0.491	20%	81
or_only / and_or_mixed	0.504 ± 0.066	0.044 ± 0.037	0.520	17%	66
negation_heavy / and_or_mixed	0.515 ± 0.076	0.049 ± 0.031	0.548	13%	57
predicateCombo					
none / single	0.467 ± 0.073	0.041 ± 0.033	0.571	0%	54
none / few	0.484 ± 0.061	0.032 ± 0.022	0.525	10%	102
none / many	0.511 ± 0.080	0.067 ± 0.036	0.595	7%	42
single / few	0.466 ± 0.053	0.039 ± 0.030	0.645	0%	117
single / many	0.476 ± 0.064	0.049 ± 0.036	0.537	13%	54
few / many	0.521 ± 0.059	0.043 ± 0.034	0.535	10%	102
paramType					
none / small	0.480 ± 0.074	0.050 ± 0.041	0.628	3%	54
none / medium	0.458 ± 0.074	0.056 ± 0.037	0.540	13%	96
none / large	0.521 ± 0.062	0.055 ± 0.037	0.569	3%	48
small / medium	0.513 ± 0.052	0.030 ± 0.025	0.583	3%	108
small / large	0.486 ± 0.072	0.046 ± 0.031	0.573	10%	63
medium / large	0.493 ± 0.068	0.036 ± 0.028	0.531	20%	102
Overall: ✓ PASS (all 18 contrasts)					

TABLE VIII
Unsupervised clustering attack (K-Means, K=number of classes / K=2 for pairs) on the on-chain gas features. ARI and NMI near 0 mean the natural gas clusters carry no information about the hidden label; NMI is interpreted against the null simulation in Section V-G. PCH clusters all executions globally (no layering).

Property	Target	n	#cls.	ARI	NMI
PCH	DUO terms	2005	6	0.0002	0.0036
PCH	ICD-10 chapters	920	10	−0.0010	0.0181
PI	policy pairs (30v30)	133	2	0.0015	0.0136
SPP	logicStructure	1141	4	−0.0007	0.0022
SPP	nestingDepth	1141	4	−0.0002	0.0018
SPP	predicateCount	1141	4	−0.0008	0.0015
SPP	setSize	1141	4	−0.0013	0.0019

a) Removing sample-size confounds.: The clustering inputs are constructed to avoid degenerate measurements. The naive PI pairs were severely imbalanced (executions in a 1:30 ratio), under which K-Means trivially fails and a low ARI/NMI would be meaningless. We rebuild the pairs into balanced 30-vs-30 contrasts, and guard the result with a permutation baseline—NMI under shuffled labels, where no true signal can exist—reporting the debiased NMI (observed minus baseline) and its empirical p-value. The conclusion is unchanged (Section V-D): the indistinguishability is a property of the protocol, not of a degenerate measurement. The ablation in Section V-H supplies the positive control: clustering significance appears on the unpadded gas transcript and disappears after normalization.

G. Sample-size sensitivity of NMI

The clustering cross-check (Section V-F) treats ARI as the primary conservative indicator because NMI can take moderately high values under small samples. To justify that choice, we quantify how finite-sample effects alone can inflate NMI even when there is no discriminative signal, using an idealized null in which all classes are statistically indistinguishable.

a) Null hypothesis.: We sample every class from the same distribution, $x_i \sim \mathcal{N}(0, I_d)$ for all classes, where d is the synthetic feature dimension and I_d is the d-dimensional identity covariance. Since no class carries class-specific information, a clustering algorithm should not recover class membership, and both NMI and ARI should concentrate around 0 as the sample size grows.

b) Simulation.: For each configuration we run $T = 1000$ independent trials. In each trial we (i) generate C classes with n i.i.d. samples per class from $\mathcal{N}(0, I_d)$, (ii) apply K-Means with $K = C$ to the pooled data, and (iii) score the predicted assignments against the ground-truth labels with NMI and ARI. We sweep n while holding C fixed, and report the trial means $\overline{\text{NMI}}$ and $\overline{\text{ARI}}$. We consider a PI-like binary structure ($C = 2$, $n \in \{3, 10, 50, 100\}$) and an ICD10-like multi-class structure ($C = 5$, $n \in \{3, 10, 100\}$).

c) Results.: Table IX shows that, although all classes are indistinguishable by construction, NMI is noticeably positive at very small n but collapses toward zero as n grows, whereas ARI stays near zero throughout. For the binary structure, $\overline{\text{NMI}}$ falls from 0.1808 at n=3 (N=6) to 0.0404 at n=10, and to 0.0066/0.0038 at n=50/100, while $\overline{\text{ARI}}$ stays at −0.0190, −0.0020, −0.0011, 0.0002. For the multi-class structure the same trend holds: $\overline{\text{NMI}}$ goes from 0.4168 (n=3) to 0.1249 (n=10) to 0.0105 (n=100), with $\overline{\text{ARI}}$ near zero (0.0005, 0.0018, −0.0000). Hence moderately high NMI in small-sample clustering should

not be read as actionable leakage, and ARI is the more conservative indicator of practical recoverability in our unsupervised evaluation.

TABLE IX
Sample-size sensitivity of NMI and ARI under the null hypothesis where all classes are i.i.d. sampled from $\mathcal{N}(0, I_d)$ (T=1000 trials per row).

Structure	n (per class)	C	N	$\overline{\text{NMI}}$	$\overline{\text{ARI}}$
PI-like	3	2	6	0.1808	−0.0190
PI-like	10	2	20	0.0404	−0.0020
PI-like	50	2	100	0.0066	−0.0011
PI-like	100	2	200	0.0038	0.0002
ICD10-like	3	5	15	0.4168	0.0005
ICD10-like	10	5	50	0.1249	0.0018
ICD10-like	100	5	500	0.0105	−0.0000

H. Ablation: gas-normalization positive control

The main results show that the full mechanism hides content, policy identity, and structure/parameters. The ablation asks why: how much of the residual attack surface is removed by randomized access-gas padding after the fixed circuit and commitment-only interface have already eliminated calldata, proof-size, and event-log channels.

We compare two aligned protection profiles on the same selected policy set. B1 uses the fixed circuit, commitment-only interface, and TPS execution profile without commit/access gas padding. B2 is the full mechanism used in the main evaluation, with randomized access-gas padding. This is the positive control for the gas channel: calldata, proof size, and event-log shape are already fixed, so B1 is an un-padded gas-transcript control rather than a run with every normalization layer removed. The same features, splits, and classifiers produce well-above-chance AUCs on B1 when padding is absent, while B2 returns the attacker toward chance. Thus the main privacy claim rests on a real ablation contrast rather than on a synthetic label derived from an input feature. The paired leakage metric is

$$\Delta_{\text{leak}} = |\text{AUC}_{\text{B1}} - 0.5| - |\text{AUC}_{\text{B2}} - 0.5|.$$

Positive values mean padding moved the attacker closer to chance, and we bound each Δ_{leak} with a 500-iteration bootstrap 95% confidence interval. Table X is intentionally separated from the main privacy verdicts: the main verdicts answer whether the full mechanism is private; this ablation attributes that privacy to a specific defense layer and reports its gas cost.

a) Padding removes otherwise-recoverable signal.: With padding disabled, attacks that are non-informative on the deployed system become measurably effective. The random-forest distinguisher on B1 reaches AUC up to 0.81 on SPP parameter-magnitude contrasts (paramType none/small) and 0.76 on structural contrasts (and_only/negation_heavy), while the corresponding B2 AUCs fall back to 0.41–0.47; the bootstrap Δ_{leak} is positive with a 95% CI that excludes zero. The clustering attacker shows the same pattern: PI pairwise clustering reaches debiased significance on 68% of pairs under B1 (mean ARI 0.50) versus 5% under B2 (mean ARI 0.0015), and SPP logic-structure clustering drops from ARI 0.067 (B1) to −0.006 (B2). This establishes randomized access-gas padding as the defense layer responsible for the gas-channel protection reported in Sections V-C–V-F, rather than weak attacks or insufficient sample size.

TABLE X
Gas-padding ablation on Polygon Amoy. B1 (no commit/access gas padding) and B2 (deployed, randomized access-gas padding) are aligned on the same policy set. Supervised AUC is the calibrated random-forest score; clustering reports K-Means ARI. $\Delta_{\text{leak}} = |\text{AUC}_{\text{B1}} - 0.5| - |\text{AUC}_{\text{B2}} - 0.5| > 0$ means padding moved the attacker toward chance; Δ_{leak} rows show the bootstrap mean with 95% CI (500 iterations). Padding collapses every recoverable signal to the chance baseline.

Attack	Target	B1 (unpad.)	B2 (pad.)
Supervised (RF AUC; Δ_{leak} with 95% CI)			
SPP	paramType none/small	0.812	0.414
	$\Delta_{\text{leak}} = 0.127$ CI$[0.055, 0.201]$		
SPP	structure and/neg-heavy	0.759	0.465
	$\Delta_{\text{leak}} = 0.195$ CI$[0.173, 0.217]$		
PCH	DUO POA	0.616	0.491
	$\Delta_{\text{leak}} = 0.092$ CI$[0.071, 0.114]$		
PI	controlled_difference	0.611	0.515
	$\Delta_{\text{leak}} = 0.102$ CI$[0.076, 0.129]$		
Unsupervised (K-Means ARI)			
PCH	ICD-10 chapter	0.064	−0.005
SPP	logic structure	0.067	−0.006
SPP	predicate count	0.054	0.001
Gas cost (mean, B1 → B2)			
Commit gas		216,562	303,556 (+40.2%)
Access gas		546,876	753,310 (+37.7%)

b) Cost.: Padding is not free: it raises mean commit gas from 216,562 (B1) to 303,556 (B2), a 40.2% increase, and mean access gas from 546,876 to 753,310, a 37.7% increase. This overhead is the principal efficiency tradeoff of the design. Table XI adds a single-point DEPTH=20 cost characterization. Gas, logs, and transaction objects are taken from the Polygon Amoy executed datasets; transaction hashes are stored in the B1/B2 executed datasets. The captured transaction dataLength values are reported instead of accessCalldataLength, which is null in these artifacts. Proving time is local prover timing from the B1 run, not an on-chain measurement. Cost-vs-DEPTH scaling is left to future work because only DEPTH=20 is compiled and evaluated here.

c) Scope.: B2 reuses the existing deployed Polygon Amoy run, so B1 and B2 are non-contemporaneous; blockDifference is excluded from every attack model to avoid confounding the comparison with timing rather than policy-dependent structure.

TABLE XI
Single-point cost profile at DEPTH=20. Gas, logs, and transaction objects are measured from Polygon Amoy executed datasets; proving time is local prover timing from the B1 run. Transaction data length reports captured dataLength values because accessCalldataLength is null in these artifacts.

Metric	Value / source
Circuit size	75,097 R1CS constraints; 74,834 wires; DEPTH=20
Public interface	2 public inputs plus 1 output
Verifier gas, unpadded B1	Commit 216,562; access 546,876 mean gas
Verifier gas, padded B2	Commit 303,556; access 753,310 mean gas
Event logs	Commit: 1 log / 180 bytes; access: 2 logs / 296 bytes
Transaction data length	B1 unique values: 330 and 0 bytes; B2 unique values: 394 and 0 bytes
Local witness time	190.8 ms mean (B1)
Local prove time	3881.5 ms mean (B1)

VI. Discussion

The evaluation indicates that TPS-ZKPI removes recoverable policy-dependent signals from normalized public-chain artifacts. Across all PCH, PI, and SPP tasks, supervised classifiers stay below the multi-seed permutation-test leakage criterion, and clustering ARI is statistically indistinguishable from zero. The gas-padding ablation (Section V-H) sharpens this conclusion: the same attacks become measurably effective once normalization is disabled, which attributes the protection specifically to transcript normalization rather than to weak attacks or insufficient sample size. Together with fixed-length policy representation, reusable interpretation, and commitment binding, these mechanisms limit the extent to which public-chain observers infer hidden policy content or structural properties from authorization transactions.

These results support a scoped privacy claim. The evaluation focuses on a metadata-only public-chain adversary that observes commitments, proofs, public parameters, calldata, events, and gas-related artifacts, but not TPS-side witness construction or off-chain policy material. Within this scope, transcript normalization eliminates the recoverable policy-dependent signals we are able to measure across PCH, PI, and SPP, at a gas-cost overhead of roughly 38–40%. We do not claim distinguishability is information-theoretically zero; rather, no modeled attacker recovers it above the chance baseline. Stronger adversaries, such as compromised proof services, adaptive requester-side probing, or off-chain side channels, remain outside the current evaluation scope and motivate the reduced-trust extensions discussed below.

The TPS trust assumption is the main architectural tradeoff. By moving confidentiality-sensitive proof generation to the TPS, TPS-ZKPI avoids exposing policy-bound proving material to requesters or public smart contracts, while preserving on-chain verifiability through zero-knowledge proof verification. However, this shifts part of the confidentiality burden to the service layer. The current provider-operated instantiation also leaves liveness and accountability unchanged: the TPS or provider can refuse to produce a proof, which may be indistinguishable from a legitimate denial at the on-chain interface. Future work should investigate accountable proof obligations, such as on-chain commitments to serve requests plus timeout-with-evidence mechanisms, as well as reduced-trust TPS designs. A distributed proof service is essentially a collaborative-SNARK direction in which no single service sees both inputs, while a TEE-backed service makes the word "trusted" concrete through hardware attestation.

VII. Conclusion

This paper studied policy-private consent enforcement in blockchain-based healthcare data sharing under public-chain observability and an explicit TPS trust assumption. We proposed TPS-ZKPI, a Trusted Proof Service-assisted zero-knowledge approach that binds hidden provider-defined policies to on-chain commitments, evaluates Boolean-based consent policies through a reusable policy interpreter, and normalizes public verification artifacts to reduce metadata-level leakage. Across supervised and unsupervised attacks on PCH, PI, and SPP, no modeled adversary recovered hidden policy information above the chance baseline under a multi-seed permutation criterion, and a paired gas-padding ablation attributed this protection specifically to gas-transcript normalization—at a 38–40% gas overhead. Future work will extend the analysis to stronger adaptive adversaries, off-chain leakage, richer policy languages, reduced-trust proof generation, and privacy-preserving resource discovery.

References

[1] P. V. Kakarlapudi and Q. H. Mahmoud, "A systematic review of blockchain for consent management," in Healthcare, vol. 9, no. 2. MDPI, 2021, p. 137.

[2] C. Welzel, M. Ostermann, H. L. Smith, T. Minssen, T. Kirsten, and S. Gilbert, "Enabling secure and self determined health data sharing and consent management," npj Digital Medicine, vol. 8, no. 1, p. 560, 2025.

[3] K. Li, A. R. Sai, and V. Urovi, "Do you need a blockchain in healthcare data sharing? a tertiary review," Exploration of Digital Health Technologies, vol. 2, no. 3, pp. 101–123, 2024.

[4] F. Béres, I. A. Seres, A. A. Benczúr, and M. Quintyne-Collins, "Blockchain is watching you: Profiling and deanonymizing ethereum users," in 2021 IEEE international conference on decentralized applications and infrastructures (DAPPS). IEEE, 2021, pp. 69–78.

[5] K. Li, A. Lohachab, M. Dumontier, and V. Urovi, "Privacy preservation in blockchain-based healthcare data sharing: A systematic review," Peer-to-Peer Networking and Applications, vol. 18, no. 6, pp. 1–53, 2025.

[6] J. Lawson, M. N. Cabili, G. Kerry, T. Boughtwood, A. Thorogood, P. Alper, S. R. Bowers, R. R. Boyles, A. J. Brookes, M. Brush et al., "The data use ontology to streamline responsible access to human biomedical datasets," Cell Genomics, vol. 1, no. 2, 2021.

[7] A. Rondelet and M. Zajac, "Zeth: On integrating zerocash on ethereum," arXiv preprint arXiv:1904.00905, 2019.

[8] Z. J. Williamson, "The aztec protocol," URL: https://github.com/AztecProtocol/AZTEC, 2018.

[9] M. Nadler and F. Schär, "Tornado cash and blockchain privacy: a primer for economists and policymakers," Federal Reserve Bank of St. Louis Review, 2023.

[10] L. Ibraimi, M. Petkovic, S. Nikova, P. Hartel, and W. Jonker, "Ciphertext-policy attribute-based threshold decryption with flexible delegation and revocation of user attributes," IEEE Trans. Image process, 2009.
[11] S. S. Ullah, V. Oleshchuk, and H. S. G. Pussewalage, "A survey on blockchain envisioned attribute based access control for internet of things: Overview, comparative analysis, and open research challenges," Computer Networks, vol. 235, p. 109994, 2023.
[12] L. Zhou, A. Diro, A. Saini, S. Kaisar, and P. C. Hiep, "Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities," Journal of Information Security and Applications, vol. 80, p. 103678, 2024.
[13] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in 2007 IEEE symposium on security and privacy (SP'07). IEEE, 2007, pp. 321–334.
[14] G. Ateniese, K. Fu, M. Green, and S. Hohenberger, "Improved proxy re-encryption schemes with applications to secure distributed storage," ACM Transactions on Information and System Security (TISSEC), vol. 9, no. 1, pp. 1–30, 2006.
[15] E. B. Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, "Zerocash: Decentralized anonymous payments from bitcoin," in 2014 IEEE symposium on security and privacy. IEEE, 2014, pp. 459–474.
[16] N. Wu, L. Xu, and L. Zhu, "A blockchain based access control scheme with hidden policy and attribute," Future Generation Computer Systems, vol. 141, pp. 186–196, 2023.
[17] R. Hu, Z. Ma, L. Li, P. Zuo, X. Li, J. Wei, and S. Liu, "An access control scheme based on blockchain and ciphertext policy-attribute based encryption," Sensors, vol. 23, no. 19, p. 8038, 2023.
[18] D. Boneh, A. Sahai, and B. Waters, "Functional encryption: Definitions and challenges," in Theory of Cryptography Conference. Springer, 2011, pp. 253–273.
[19] Secret Network, "Secret network: A privacy-preserving secret contract platform," White paper, 2020, available online.
[20] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, and D. Song, "Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts," in 2019 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2019, pp. 185–200.
[21] C. A. Ardagna, S. De Capitani di Vimercati, S. Paraboschi, E. Pedrini, and P. Samarati, "An xacml-based privacy-centered access control system," in Proceedings of the first ACM workshop on Information security governance, 2009, pp. 49–58.
[22] M. Sheikhalishahi, G. Tillem, Z. Erkin, and N. Zannone, "Privacy-preserving multi-party access control," in Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society, 2019, pp. 1–13.
[23] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, 2019.
[24] M. Campanelli, D. Fiore, and A. Querol, "Legosnark: Modular design and composition of succinct zero-knowledge proofs," in Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 2075–2092.
[25] T. Liu, Z. Zhang, Y. Zhang, W. Hu, and Y. Zhang, "Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine," Journal of Cryptology, vol. 38, no. 2, p. 17, 2025.
[26] T. Miller, B. Alvarez, and T. Hoang, "Vtbc: Privatizing the volume and timing of transactions for blockchain applications," in 2023 32nd International Conference on Computer Communications and Networks (ICCCN). IEEE, 2023, pp. 1–10.